Sarabo — Presentation Script (full version, for rewriting)

This is the **long version** (about 1,900 words, around 13–14 minutes if read in full). It has more than you need on purpose, so you can cut it down to your own 5-minute script. Each slide has:

- **Core** — the sentences to keep (these are the same ideas as the 5-minute script).
- **Extra** — more detail, examples and numbers you can swap in, or use in the Q&A.
- **Transition** — one line to move to the next slide.

Slide 1 — Title

Core.
Assalamu alaikum, and good morning to the respected board members. I am Jahid Hasan, ID 222-115-116, from Batch 57, section C, Department of Computer Science and Engineering, Metropolitan University, Sylhet. My final-year project, supervised by Shrabanti Chowdhury, Senior Lecturer, is **Sarabo — an electronics repair service management platform.** The tagline on the slide says what it is for: *repair your device with confidence*.

Extra.
I built the whole system myself: the requirements, the design, the server, the web client, the tests, the deployment and the report. It is not a prototype on my laptop — it is deployed and running on the internet today.

Transition. Let me start with why this project is needed.

Slide 2 — The problem

Core.
Getting a device repaired today is a leap of faith. There are three people in this story, and each one has a problem.
The **customer** hands over a phone or a laptop without knowing the final price, and once it is gone, they have no idea where it is or what is happening to it.
The **technician** may be skilled, but finds work only by word of mouth, and is never sure they will actually be paid.
And **whoever runs the service** cannot see which jobs are stuck, and has to handle refunds by hand.
The result is low trust on both sides — and many repairs simply never happen.

Extra.

- Think of a cracked phone screen. The customer asks "how much?" and hears "we'll see after opening it". There is no written quote, no record of the phone's condition at hand-over, and no warranty on paper.
- If something goes wrong — the shop says the phone was already damaged, or the repair fails after a week — there is no evidence and no process.

- For technicians, especially freelancers, the problem is the opposite: they have the skill but no steady flow of jobs and no protection if a customer does not pay.

Transition. So the question I asked was: can one system make this process clear and fair for all three?

Slide 3 — The solution

Core.
Sarabo turns this into **one tracked, priced workflow for three roles.**
The **customer** can request a repair, compare offers, pay and track the device.
The **technician** can find jobs, send a quote, do the repair and get paid.
The **admin** approves technicians, sets prices, handles problems and reads reports.
The catalogue has **45 repair services across 8 device types**, all priced in Bangladeshi taka, and the system is **live on Vercel.**

Extra.

- Every request moves through clear stages — Request, Inspect, Approve, Repaired — shown as a progress bar on every request page, with a "Your next step" panel so each person always knows what to do.
- A customer who is not signed in can still track a request with a private tracking code; the public tracking page shows only the stage and time, never a name, address or payment.
- If no approved technician covers a region for that repair, Sarabo refuses the request politely at the start instead of leaving the customer waiting — and counts it, so the admin can see where technicians are needed.

Transition. Let me walk you through the full journey of one repair.

Slide 4 — How it works

Core.
The full journey has six steps.
One — Request. The customer describes the problem, even in Banglish like "screen venge gese", and AI fills in the form. The customer also picks a pickup time.
Two — Offers. Local technicians with the right skill see the job and send offers.
Three — Book. The customer chooses one and pays a small inspection fee to book that technician.
Four — Quote. After inspecting the device, the technician sends an itemised quote: parts, labour and any extra charge.
Five — Pay and repair. The customer approves and pays only what is left, because the inspection fee is already counted.
Six — Hand-over. The device is returned, the technician receives 90 percent, Sarabo keeps 10 percent, and a 30-day warranty starts.
It is **fair by design**: if the technician does not show up, or the customer cancels at least two hours early, the fee is fully refunded. Otherwise the fee is split 90/10.

Extra.

- Pickup slots: four two-hour windows a day between 10 am and 6 pm, closed on Friday, up to 7 days ahead, at least one hour in advance, and limited per region so technicians are not overloaded.
- The minimum inspection fee is 100 taka, or 200 taka for bigger repairs, so a technician never travels for free.
- At pickup the technician records the device's condition, and the customer can dispute it — this record protects both sides later, for example in a warranty claim.
- No money is charged when a request is created. The customer pays nothing until they choose a technician.
- The technician's money is held until the customer confirms they received the device, so the technician is guaranteed payment and the customer is protected.

Transition. Behind this journey is the data model.

Slide 5 — ER diagram

Core.
This is the entity-relationship diagram of the data in MongoDB.
A **user** creates many repair requests, and a user can also become a **technician**.
Each **repair request** is priced by one **service**, receives many **offers**, and has its **payments**, at most one **review**, and at most two **warranty claims.**
The quote and the inspection are stored inside the request itself, because they only belong to that request.
The double bar means "one" and the crow's foot means "many".

Extra.

- I chose MongoDB because one repair request, with its quote, inspection, history and settlement, naturally fits one document.
- Where data across collections must stay consistent — for example a payment and the booking it creates — I use multi-document transactions.
- Prices are copied into the request when it is created (a price snapshot), so if the admin changes a price later, old requests are not affected.
- The technician's wallet balance is **not stored**; it is calculated every time from settlements and withdrawals, so it can never drift out of sync.

Transition. Now let me show you the key screens, starting with the customer.

Slide 6 — Customer: ranked offers and the quote

Core.
When offers arrive, Sarabo **ranks them** using five things it already stores: the technician's rating, reliability, price, district and reply time. The reasons appear as small tags such as "Best match" or "Same district". But the customer still makes the final choice.
After the inspection, the quote shows exactly what is left to pay. In this example, the quote is 4,000 taka, the 200-taka inspection fee is already paid, so only **3,800 taka** is left. Approving and paying happen in one step.

Extra.

- Weights: rating 30 percent, reliability 25, price 20, district 15, reply time 10.
- A new technician gets a neutral score — not punished and not boosted — so new people can still get work.
- Ratings are smoothed, so one five-star review cannot beat forty reviews at 4.8.
- If the customer declines the quote, the technician can send one revised quote; otherwise the request closes and the fee is kept, split 90/10.
- After payment, the customer can download a PDF receipt.

Transition. What happens after the repair is just as important.

Slide 7 — Warranty, and no stuck repairs

Core.
Every repair comes with a **30-day warranty.** If the same fault comes back, the customer opens a claim, the same technician re-inspects it for free, and a covered fault is either repaired again or refunded.
I also made sure that **no request can wait forever.** Every waiting state has a deadline, an automatic step or an admin alert. An unpaid approved quote closes after 3 days. If the customer does not confirm receipt, it is confirmed automatically after 3 days. If a technician can't finish a repair, the customer is refunded. And stalled work raises an alert for the admin.

Extra.

- Warranty limits: at most two claims per repair and one open at a time. The technician has 3 days to book a re-inspection, 3 days to decide and 7 days to re-repair; if they miss a deadline, the admin takes over.
- "Not covered" — for example new water or drop damage — must have a reason, and the customer can ask the admin to review it. The pickup condition record is the evidence.
- If a refund happens after the technician already withdrew the money, the amount is shown as owed and taken from their next earnings.
- A daily scheduled job runs reminders and all these deadlines; each job runs separately, so if one fails, the others still run.

Transition. Some of these steps are helped by AI.

Slide 8 — Applied AI (Google Gemini)

Core.
Google Gemini helps in four places. The **request helper** turns the customer's own words into a filled form. The **offer note** helps a technician write a clear message, in English or Bangla. The **support chat** answers questions about services and prices. And the **report summary** explains the admin's weekly numbers in plain sentences.
But the rule is simple: **AI suggests, people decide.** AI never sets a price. Every answer is checked against live data. No personal data is sent. And usage has daily limits.

Extra.

- The request helper can only pick services that really exist in the live catalogue; the server checks every suggestion again, and the customer can change everything or press Undo.
- Phone numbers, emails and links are removed from AI offer notes, so customers and technicians cannot move the deal outside the platform.
- In the report summary, any sentence that contains a number not found in the real data is removed.
- Every AI feature is optional. If AI is slow or switched off, the user just fills the form by hand.
- I measured the models: the current model answers in about 1.4 to 1.9 seconds, with a second model as a fallback.

Transition. None of this is safe unless the foundation is solid.

Slide 9 — Architecture

Core.
A **React 19** web client talks to an **Express 5 API** on Vercel, in the Mumbai region, right next to the **MongoDB Atlas** database with 21 collections. Around it are Firebase for sign-in, Stripe for payments, Supabase for photos, Gemini for AI and Gmail for email.
Because this system handles money, I followed three rules.
The server decides — role and current state are checked on every action, never trusted from the browser.
All-or-nothing — booking, payment and refunds run inside database transactions.
Exactly once — no double payment and no double booking, even if two people click at the same moment.

Extra.

- Payment is confirmed only by Stripe's signed webhook, not by the success page, and the same event arriving twice is recorded once.
- If a payment arrives but the job was already taken, nothing is booked and the fee is refunded automatically.
- Damage photos are in a private bucket and are shown only through links that expire in a few minutes.
- Moving the API to Mumbai, next to the database, cut the first request from 10.7 seconds to 2.9, and normal requests from 0.75 to 0.2 seconds.
- Secrets live only in server environment variables, never in the code or the browser.

Transition. So, how do I know it works?

Slide 10 — Results, limitations and next steps

Core.
All **2,761 server tests** and all **58 browser tests** pass, all **twelve objectives** were met, and the system is **live.**
I want to be honest about the limits: Stripe runs in test mode, bKash payouts to technicians are sent manually by the admin, and there are no real customers yet.
Next, I would add a local payment gateway such as SSLCOMMERZ or bKash, and a mobile app with a full

Bangla interface.
Thank you. Now I will show you the live demo.

Extra.

- The server tests use real HTTP calls against a real MongoDB replica set, with fake Stripe, AI and storage, and they include race tests such as two admins acting at once.
- The browser tests run on both desktop and a mobile screen size, and every push runs all tests automatically on GitHub Actions.
- Other limits worth mentioning if asked: one active job per technician at a time, matching by district and region instead of live GPS, no SMS (it is paid), and no load or security audit yet.

How to cut this down to 5 minutes

1. Keep only the **Core** paragraphs — that alone is about 1,000 words (≈7 minutes).
2. Remove one sentence of examples from slides 2, 3 and 9 and the pickup-time sentence on slide 4.
3. Target about **700 words** total at around 140 words per minute.
4. Use the slide timings from `script-5min.md`: 0:00, 0:15, 0:40, 1:05, 1:45, 2:20, 3:00, 3:35, 4:05, 4:35.
5. Move any **Extra** point you like into your Q&A answers instead.